

Lecture 5 — Association, Aggregation, and Composition

- Goal: Understand how classes relate to one another using 'has-a' relationships.
- Today we'll explore Association, Aggregation, and Composition — their meanings, examples, and differences.
- We'll also compare them with inheritance and build practical examples step by step.

Learning Objectives

- Understand the difference between 'is-a' and 'has-a' relationships.
- Explain Association, Aggregation, and Composition with real-world analogies.
- Implement each relationship in Python classes.
- Identify when to use each type of relationship in software design.

Why Relationships Between Classes?

- Large systems require objects to work together.
- Inheritance ('is-a') helps share structure and behavior.
- But when one class uses or owns another → 'has-a' relationships.
- These relationships improve modularity and reuse.

Types of Relationships in OOP

- 1■■ Association — one object uses another (loose connection).
- 2■■ Aggregation — one object contains others (independent lifetime).
- 3■■ Composition — one object owns others (dependent lifetime).

Association — Basic Concept

- Association means one class uses another class.
- No ownership or lifetime dependency — both can exist independently.
- Example: A Doctor treats many Patients; both exist independently.

Association Example — Doctor and Patient

- Doctor 'has-a' reference to a Patient (association).
- Patient and Doctor are created independently.
- Doctor only uses the Patient temporarily.

```
class Patient:
    def __init__(self, name):
        self.name = name

class Doctor:
    def __init__(self, name):
        self.name = name

    def treat_patient(self, patient):
        print(f"Dr. {self.name} is treating {patient.name}")

# Example usage
p1 = Patient("Ali")
d1 = Doctor("Sara")
d1.treat_patient(p1)
```

Aggregation — Basic Concept

- Aggregation is a stronger 'has-a' relationship.
- The parent (container) holds references to objects, but doesn't own them.
- Parts can exist even if the container is destroyed.
- Example: A Course has many Students, but students exist even without the course.

Aggregation Example — Course and Students

```
class Student:
    def __init__(self, name):
        self.name = name

class Course:
    def __init__(self, title):
        self.title = title
        self.students = []

    def add_student(self, student):
        self.students.append(student)

    def list_students(self):
        for s in self.students:
            print(f"{s.name} enrolled in {self.title}")

# Example usage
s1 = Student("Ali")
s2 = Student("Sara")
c1 = Course("Data Structures")
c1.add_student(s1)
c1.add_student(s2)
c1.list_students()
```


Composition Example — Car and Engine

- Engine is created inside Car's constructor.
- Car owns the Engine → strong lifetime dependency.
- Engine cannot exist without Car.

```
class Engine:
    def __init__(self, horsepower):
        self.horsepower = horsepower

    def start(self):
        print(f"Engine with {self.horsepower} HP started")

class Car:
    def __init__(self, brand, horsepower):
        self.brand = brand
        self.engine = Engine(horsepower)

    def start_car(self):
        print(f"{self.brand} car starting...")
        self.engine.start()

# Example usage
car1 = Car("Toyota", 150)
car1.start_car()
```

Comparison Table

- Association → Loose connection, no ownership.
- Aggregation → Container has parts, but parts live independently.
- Composition → Strong ownership; part dies with parent.

Practice Problem — Order and Items (Composition)

- Create an Order class that owns multiple OrderItem objects.
- Each item has name, price, and quantity.
- Order can calculate total and display all items.

```
class OrderItem:
    def __init__(self, name, price, qty):
        self.name = name
        self.price = price
        self.qty = qty

    def total(self):
        return self.price * self.qty

class Order:
    def __init__(self):
        self.items = []

    def add_item(self, name, price, qty):
        self.items.append(OrderItem(name, price, qty))

    def show_items(self):
        for i in self.items:
            print(f"{i.name}: {i.qty} x ${i.price} = ${i.total()}")

o1 = Order()
o1.add_item("Pen", 2, 3)
o1.add_item("Notebook", 5, 2)
o1.show_items()
```

Summary

- Association → Objects communicate but live separately.
- Aggregation → One object has others that live independently.
- Composition → One object owns others completely.
- Choose based on dependency and lifetime of objects.